Threat Model & Quantitative Risk Assessment

SecureApp-Sprint-Harry — CYC386 Secure Software Design and Development (Fall 2026)

Scope: Flask web application (`app.py`) and native C helper (`native/bof_helper.c`). **Methodology:** STRIDE per DFD element, attack-tree decomposition, and CVSS v3.1 scoring. **Standards:** CDF Unit 3; OWASP Testing Guide; CVSS v3.1 Specification.

1. System Overview & Assets

SecureApp is a multi-user Flask application exposing authentication (`/login`), a user-profile service (`/profile`), a network-diagnostics tool (`/ping`), a fund-transfer action (`/transfer`), and a global response filter that sets anti-framing headers. A compiled C helper performs fixed-size string processing. The in-scope assets and their primary CIA concern:

ID	Asset	Trust level	Primary CIA concern
AST-01	Session token / auth cookie (D1)	Server	Confidentiality, Integrity
AST-02	User records incl. confidential bios (D2)	Server	Confidentiality
AST-03	Ping diagnostics path (shell invocation)	Server	Integrity, Availability
AST-04	Native C helper (fixed-size buffer)	Host	Integrity, Availability
AST-05	Fund-transfer state-changing action	Server	Integrity
AST-06	Rendered UI (framing surface)	Client	Integrity

2. Context Data-Flow Diagram (Level 0)

The single trust boundary separates untrusted clients (browser, attacker) from the server-side process and the host OS/shell it invokes. All data crossing the boundary is attacker-controllable.

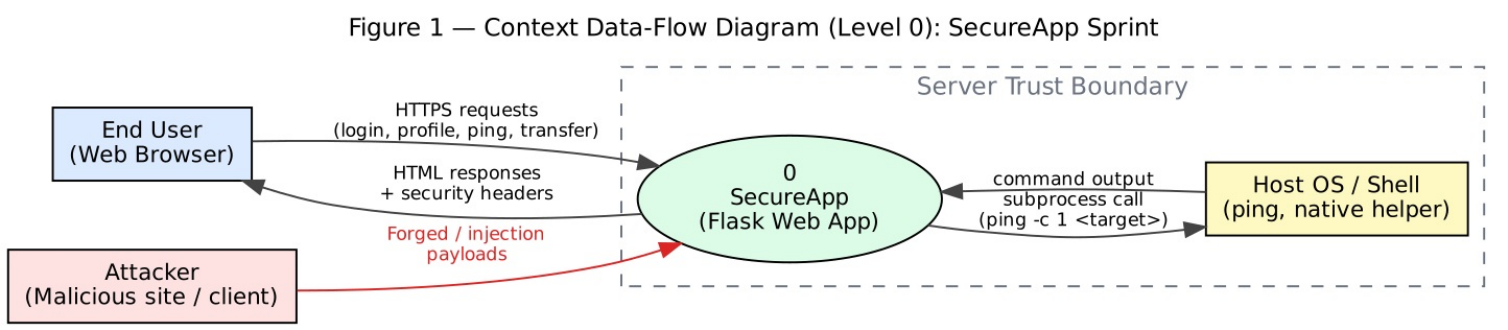


Figure 1 — Context DFD (Level 0).

3. Level-1 Data-Flow Diagram

The application process is decomposed into five sub-processes and two data stores. Red dashed edges mark the attacker-reachable flows that map to the mandatory vulnerability classes. Every crossing of the dashed Application Trust Boundary is a point where input validation and authorization must be enforced.

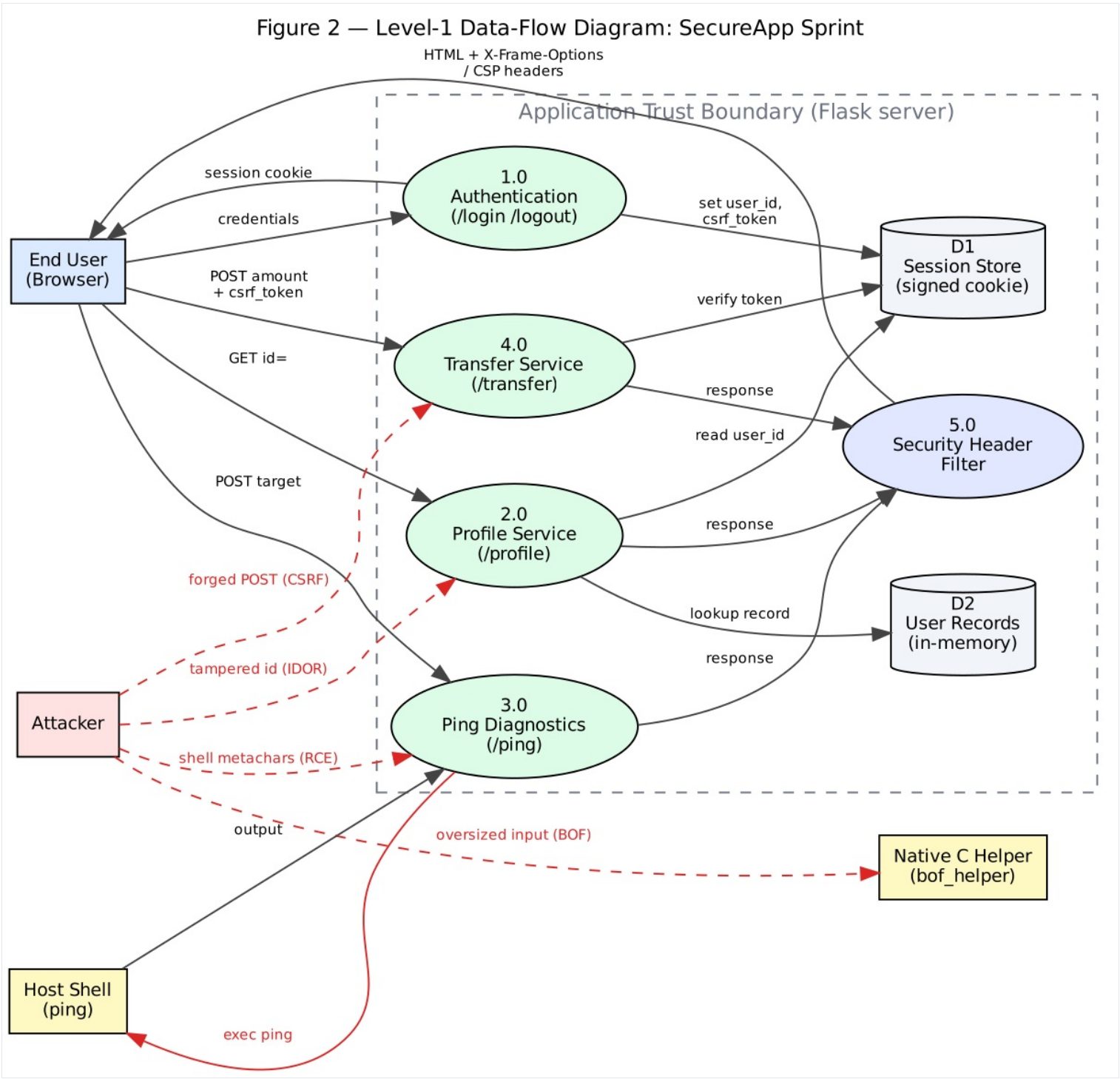


Figure 2 — Level-1 DFD with trust boundary and threat edges.

4. STRIDE Analysis (per element)

DFD element	STRIDE	Threat	CWE	Mitigation implemented
1.0 Authentication	S	Session impersonation via weak/forgeable cookie	CWE-287	Signed session cookie, HttpOnly, SameSite=Strict
2.0 Profile Service	I (Info Disclosure)	IDOR — tampered `id` reads another user's record	CWE-639	Object-level authorization (`rid == session.user_id` else 403)
3.0 Ping Diagnostics	T / E	OS command injection via shell metacharacters	CWE-78	`shell=False` argument list + host allow-list regex
4.0 Transfer Service	S / T	CSRF — forged state-changing POST	CWE-352	Synchronizer token + SameSite=Strict cookie
5.0 Security Header Filter	T (UI redress)	Clickjacking via iframe embedding	CWE-1021	X-Frame-Options: DENY + CSP frame-ancestors 'none'
Native C helper	D / E	Stack buffer overflow → crash or control-flow hijack	CWE-120	`snprintf` bound + stack canary + FORTIFY + ASLR/NX
D1 Session Store	I (Info Disclosure)	Token theft via client-side script	CWE-1004	HttpOnly flag prevents JS access
D2 User Records	R (Repudiation)	State change without audit trail	CWE-778	Server-side logging of auth/transfer events (recommended)

5. Attack Trees

Two high-impact goals are decomposed below. AND-gates require all child conditions; OR-gates require any one. Green nodes show the deployed defence that breaks the path.

Figure 3 — Attack Tree 1: Remote Code Execution on the server

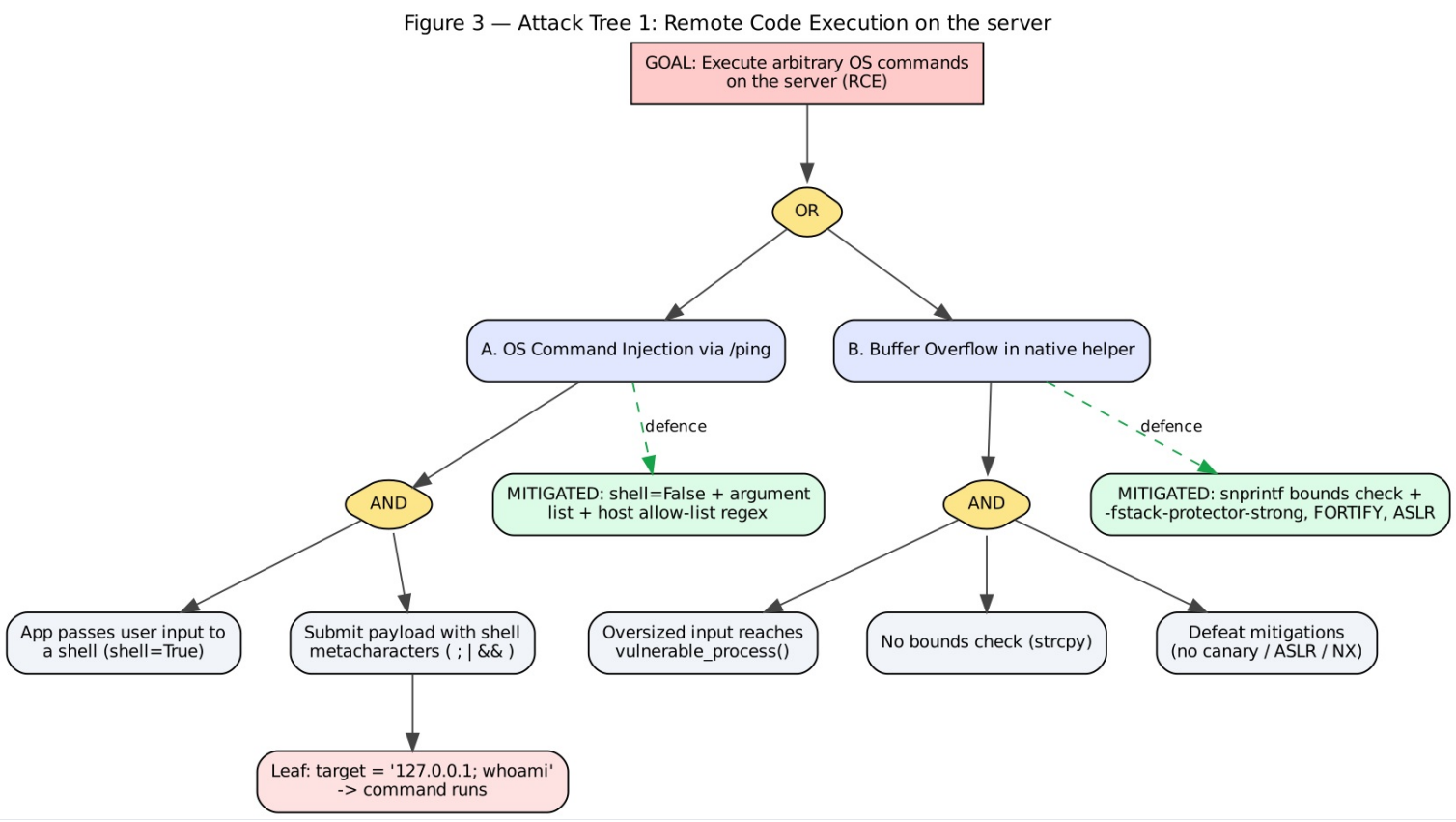


Figure 3 — Attack Tree 1: Remote Code Execution.

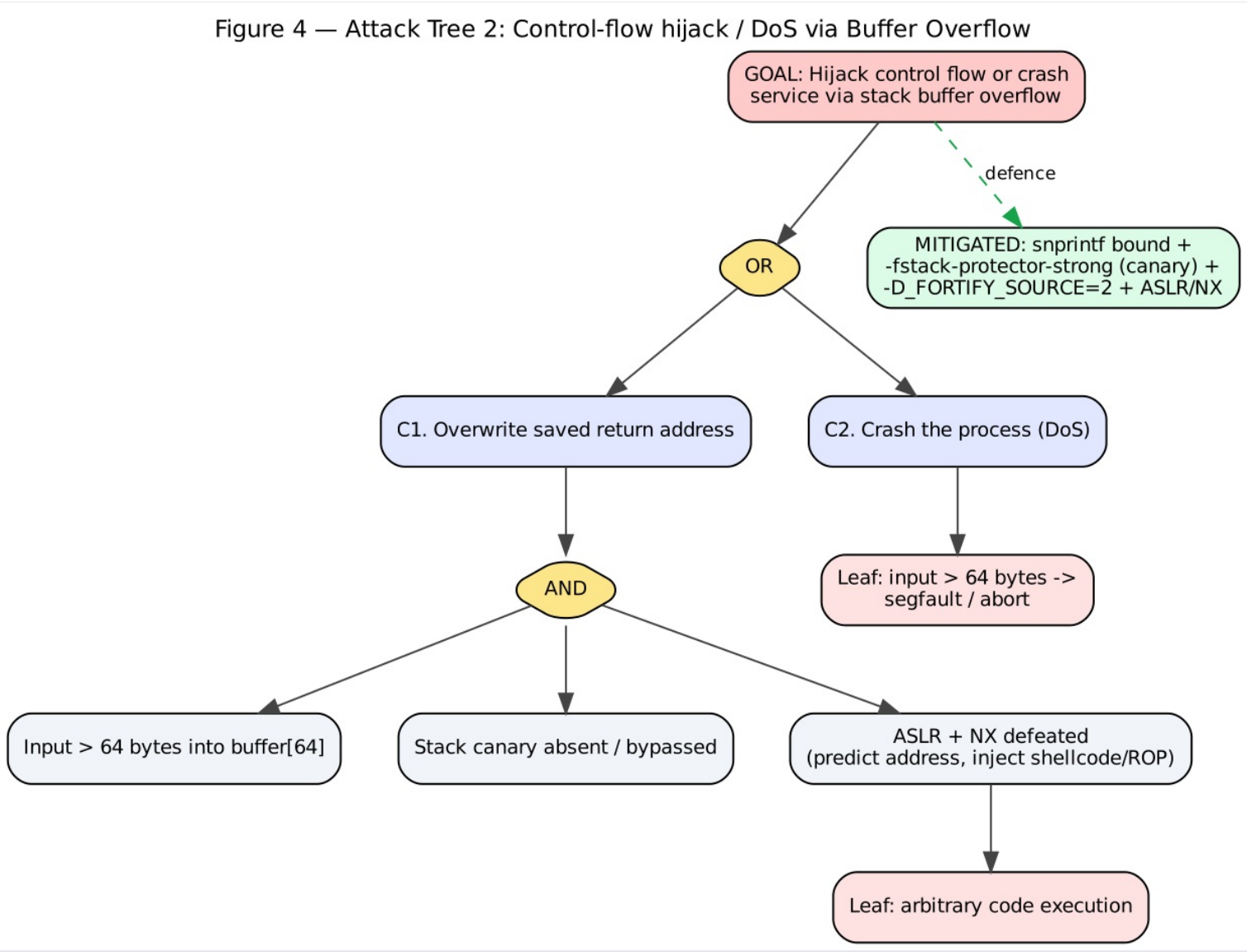


Figure 4 — Attack Tree 2: Buffer-overflow control-flow hijack / DoS.

6. CVSS v3.1 Risk Register

All High/Critical findings are scored below with justified vector strings. Severity bands follow the CVSS v3.1 qualitative scale.

Finding (CWE)	CVSS v3.1 vector	Base	Severity	Vector justification
Command Injection (CWE-78)	`AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`	9.8	**Critical**	Unauthenticated network endpoint; reliable shell execution; full host compromise.
Buffer Overflow (CWE-120)	`AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`	7.8	**High**	Local input to the helper; overflow can hijack flow or crash the process.
IDOR (CWE-639)	`AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`	6.5	**Medium**	Authenticated user reads other users' confidential records; no integrity/availability impact.
CSRF (CWE-352)	`AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N`	6.5	**Medium**	Victim must visit a malicious page; forged request performs an unauthorized state change.
Clickjacking (CWE-1021)	`AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N`	4.3	**Medium**	Requires victim interaction on a framed page; limited integrity impact.

7. Residual Risk Statement

After remediation, every mandatory finding is reduced to **Low** residual risk: the ping path no longer invokes a shell, the helper is bounds-checked and compiled with canary/FORTIFY/ASLR/NX, profile access is authorization-checked, transfers require a synchronizer token, and framing is denied by header policy. Verification is enforced continuously by the CI pipeline (unit tests + Bandit SAST gate) and by OWASP ZAP DAST.

Residual risks that remain outside the mandatory scope, with accepted controls:

Residual risk	Likelihood	Control / acceptance
Zero-day CVEs in third-party dependencies	Low	`pip-audit` in CI; scheduled dependency updates.
Host OS / kernel vulnerabilities	Low	Patch management; least-privilege runtime (future: container with dropped capabilities).
Credential phishing / weak passwords	Medium	Out of scope for this sprint; future: MFA and password policy.
Application-layer DoS (request flooding)	Medium	Future: rate limiting and request timeouts (timeouts already set on subprocess calls).